❖ **Câu lệnh For:**

for (*biến điều khiển=giá trị đầu****;*** *biến điều khiển<=giá trị cuối****;*** *biến điều khiển++)*

Lệnh;

➔ *Giải thích:* ***biến điều khiển*** *sẽ lần lượt được nhận từ* ***giá trị đầu*** *đến* ***giá trị cuối****; mỗi lần* ***biến điều khiển*** *nhận một giá trị thì* ***Lệnh*** *được thực hiện.*

Trong đó:

✓ ***biến điều khiển*** *thường là biến nguyên.*

✓ ***Lệnh****: là lệnh đơn hoặc lệnh ghép (giống phần câu lệnh if)*

- ***Ví dụ 1:***

```
for (i=1; i<=10; i++)
    cout << i << endl;
```

➔ *Biến* ***i*** *sẽ được nhận giá trị lần lượt từ* ***1*** *đến* ***10****, mỗi lần i nhận một giá trị thì lệnh* ***cout << i<<endl*** *được thực hiện. Điều đó có nghĩa là: sẽ ghi các giá trị từ 1 đến 10, mỗi giá trị trên 1 dòng.*

- ***Ví dụ 2:***

```
s=0;
for (i=1; i<=n; i++)
    s=s+i;
```

➔ *Biến* ***i*** *sẽ được nhận giá trị lần lượt từ* ***1*** *đến* ***n****, mỗi lần i nhận một giá trị thì lệnh* ***s=s+i*** *được thực hiện. Điều đó có nghĩa là: Ban đầu s có giá trị là 0; khi i=1 thì giá trị 1 sẽ được cộng thêm vào s, khi i=2 thì giá trị 2 sẽ được cộng thêm vào s,…, khi i=n thì giá trị n sẽ được cộng thêm vào s. Như vậy sau khi hoàn tất thì s=1+2+…+n.*

☞ ***Kiến thức bổ sung:***

for (*biến điều khiển=giá trị cuối****;*** *biến điều khiển>=giá trị đầu****;*** *biến điều khiển--)*

Lệnh;

❖ **Câu lệnh while…:**

while (ĐK)
Lệnh;

➔ *Giải thích: trong khi **ĐK** còn đúng*
*thì **Lệnh** được thực hiện.*

Trong đó:

- ✓ ***ĐK**: thường là một biểu thức quan hệ, logic,…*
- ✓ ***Lệnh**: là lệnh đơn hay lệnh ghép (là kết hợp nhiều lệnh đơn, được bao bởi cặp từ khóa "{… }")*
- ✓ *Trong phần **Lệnh** thì luôn có ít nhất một lệnh tác động đến **ĐK** (để ĐK dẫn tới sai) hoặc một lệnh để ngưng lặp (lệnh break) .*

- ***Ví dụ 1: Tính tổng các chữ số của số nguyên dương n.***

```
d=0;
while (n>0)
   {
      d=d+(n % 10);  {lấy chữ số tận cùng của n (n % 10) cộng vào biến d}
      n=n / 10;      {bỏ 1 chữ số tận cùng của n (n / 10) gán lại cho chính n}
   }
```

➔ Đáp án là biến d.

- ***Ví dụ 2: Tìm UCLN(a,b)***

Ý tưởng: lặp lại việc thay thế số lớn bằng hiệu của nó với số còn lại.

```
while (a!=b)
   {
      if (a>b)
         a=a-b;
      if (b>a)
         b=b-a;
   };
ucln=a;
```

- ***Ví dụ 3: Tìm số nguyên tố nhỏ nhất không nhỏ hơn n.***

Ý tưởng: Trong khi n không phải nguyên tố thì tăng n lên 1 đơn vị.

```
while (isprime(n)!=1)
   n = n+1;
```

➔ Đáp án là biến n.

*(***Chương trình có sử dụng hàm kiểm tra số n có phải là số nguyên tố hay không)*

❖ **Hàm tìm ước chung lớn nhất của 2 số:**

```
long long ucln(long long a, long long b)
    {
        while (a!=b)
        {
            if (a>b)
                a=a-b;
            if (b>a)
                b=b-a;
        };
    return a;
};
```

```
long long ucln(long long a, long long b)
    {   long long r;
        while (b!=0)
        {
            r=a % b;
            a=b;
            b=r;
        };
    return a;
};
```

❖ **Hàm tìm bội chung nhỏ nhất của 2 số:**

```
long long bcnn(long long a, long long  b)
    {
        bcnn=(a / UCLN(a,b))*b;
    };
```

❖ **Hàm kiểm tra một số nguyên có phải là số nguyên tố hay không (1 or 0):**

```
int isprime(long long n)
    {   long i;
        if (n<2)
            return 0;
        for (i=2; i<=floor(sqrt(n)); i++)
            if (n % i=0)
                return 0;
        return 1;
    };
```